

08/12/2022

Support de cours java Niveau 2

KANGA Koffi

Table des matières

A - Collection Java partie 1 : Introduction	4
B - Les collections Java partie2 - les listes : linkedList	5
1 - L'interface List.....	5
Exemple.....	5
2 - Ajout d'éléments à la liste	5
2 - 1 - Ajout d'un nouvel élément à la liste (par défaut à la fin de la liste)	5
Exemple.....	5
2 - 2 - Ajout d'un élément à une position déterminée.....	6
Exemple.....	7
3 - Récupération des élément de la liste.....	7
Exemple.....	7
4 - Suppression d'un élément de la liste	8
4 - 1 - Suppression d'un élément par son index	8
Exemple.....	8
4 - 2 - Suppression directe d'un élément par son nom	9
Exemple.....	9
4 - 3 - Vider la liste.....	9
Exemple.....	9
5 - Parcourt de la liste.....	10
5 - 1 Parcourt de la liste par la méthode listIterator()	10
Exemple.....	10
5 - 2 -Parcourt de la liste à l'aide de la boucle for	11
Exemple.....	11
Remarque :	11
C - Collections Java partie 3 - Queue.....	13
1 - L'interface Queue Java	13
14 - Graphique avec Java Swing.....	14
1 - Introduction au Bibliothèque Graphique Java Swing.....	14
2 - Première Fenêtre avec Java Swing	14
Exemple.....	16

3 - Les conteneurs Java Swing (Swing Containers)	17
4 - Les composants Java Swing	17
5 - Ajout de composants Swing à la fenêtre	17
5 - 1 - Ajout d'un bouton	17
Remarque	18
6 - Les événements Java Swing	19
Exemple avec ActionListener appliquer à JButton	19
Exemple	19
5 - Le composant JTable	20
Exemple	21
D - Java JTextArea	23
1 - Déclaration de classe JTextArea via l'héritage de JTextComponent	23
2 - Constructeur de classe & description	23
3 - Méthodes de classe & description	23
E - JMenu : Création des menus avec Java	25
1 - La barre des menus JMenuBar Java	25
a) Création d'une fenêtre dotée d'un JPanel	25
b. Création de la barre des menus avec la classe JMenuBar	25
2- La classe JMenu	25
Code Complet	25
3 - Création des sous menus avec la classe JMenuItem	27
Exemple	27
F - La liste JComboBox Java	29
1 - Création d'un JComboBox Java	29
Exemple :	30
3 - Ajout d'actions à la liste JComboBox	30
Exemple complet :	31
G - Interface graphique avec WindowBuilder	33
1 - Introduction	33
2 – Installation de WindowBuilder	34
3 - Comment utiliser WindowBuilder ?	35
H - Java IO- Gestion des fichiers en Java	38

1 - La classe File Java	38
2 - Création d'un fichier avec la classe File Java.....	38
3 - Ecrire dans un fichier existant à l'aide de la classe FileWriter.....	39
4 - Supprimer un fichier existant avec la méthode delete()	40

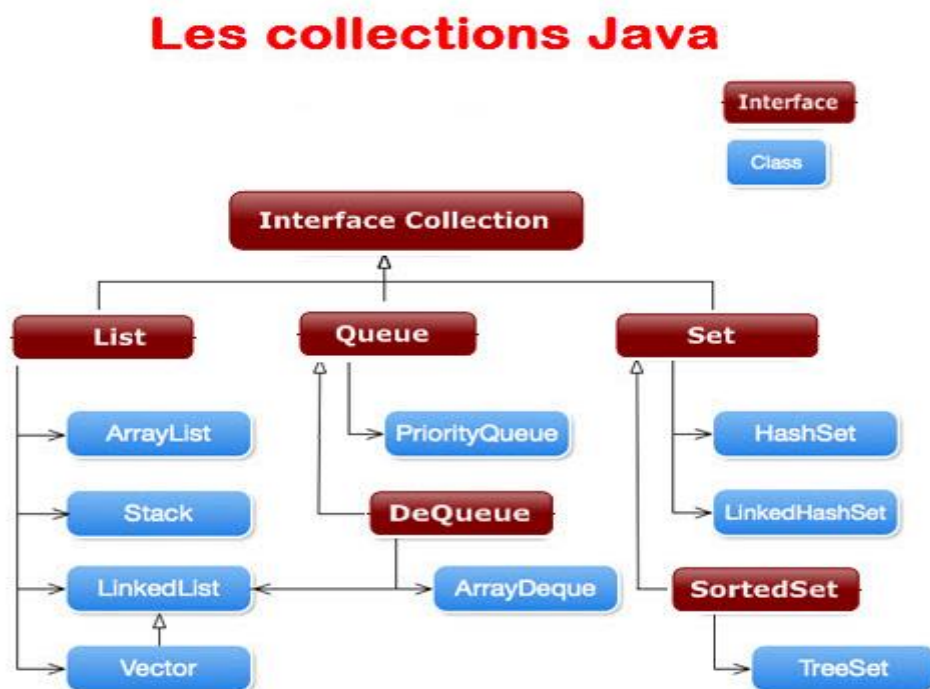
A - Collection Java partie 1 : Introduction

1 - Introduction

Afin d'organiser et améliorer l'accès aux données, Java est doté d'un ensemble d'interfaces et de classes regroupé sous le nom de **collections**. Ainsi le framework Collections Java est une collection d'interfaces et de classes qui permet de stocker et de traiter efficacement les données. Toutes les collections possèdent les mêmes opérations de base, à savoir:

- Ajout d'élément (s) à la collection
- Retrait des éléments de la collection
- Obtention du nombre d'éléments dans la collection
- Itération pour basculer d'un élément à l'autre...

Voici la hiérarchie des collections Java :



Remarque : Les interfaces sont marquées en couleur marron tandis que les classes sont marquées en bleue

Exemple : ArrayList est une classe qui implémente l'interface List et par suite on peut faire l'instanciation :

List maListe2 = new ArrayList();

Mais on ne peut plus faire l'instanciation :

List maListe2 = new ArrayList();

Puisque l'élément List est une interface qui ne peut être instanciée !

B - Les collections Java partie2 - les listes : linkedList

1 - L'interface List

LinkedList (liste chaînée) est une **implémentation** de l'interface **List**.

Nous allons découvrir ensemble , comment **créer** l'élément **linkedList**, comment **afficher** ses élément, comment lui **ajouter** des éléments avec des exemples simples et rapides.

Pour créer une **liste chaînée (linkedList)** il suffit de faire l'**instanciation** sur sa classe linkedList, sans oublier d'importer le package : **java.util**

Voici un exemple de liste chaînée (linkedList) formée d'éléments du type **String** :

Exemple

```
import java.util.*;
public class LinkedList {

    public static void main(String args[]) {

        /*Déclaration d'une liste chaînée ( Linked List ) */
        LinkedList<String> maListe = new LinkedList<String>();
    }
}
```

2 - Ajout d'éléments à la liste

2 - 1 - Ajout d'un nouvel élément à la liste (par défaut à la fin de la liste)

Pour ajouter des éléments à la liste, on utilise la syntaxe :

```
nom_de_la_liste.add("éléments");
```

Exemple

```
import java.util.*;
public class LinkedList {

    public static void main(String args[]) {

        /*Déclaration d'une liste chaînée ( Linked List ) */
        LinkedList<String> maListe = new LinkedList<String>();
```

```

        // Ajout d'éléments à la liste
        maListe.add("Voiture");
        maListe.add("Moto");
        maListe.add("Vélo");
    }
}

```

Voici maintenant le code pour afficher la liste :

```

import java.util.*;
public class LinkedList {

    public static void main(String args[]) {

        /*Déclaration d'une liste chaînée ( Linked List ) */
        LinkedList<String> maListe = new LinkedList<String>();

        // Ajout d'éléments à la liste
        maListe.add("Voiture");
        maListe.add("Moto");
        maListe.add("Vélo");
        /*Affichage de la liste chaînée*/
        System.out.println("Contenu de la liste: " + maListe);
    }
}

```

2 - 2 - Ajout d'un élément à une position déterminée

Nous allons voir maintenant comment ajouter un élément au début ou à la fin de la liste ou encore à une position donnée :

- Pour ajouter un élément au début de la liste, on utilise la syntaxe : **nom-de_la_liste.addFirst("élément au début");**
- Pour ajouter un élément à la fin de la liste, on utilise la syntaxe : **nom-de_la_liste.addLast("élément à la fin");**
- Pour ajouter un élément à une position **k** donnée, on utilise la syntaxe : **nom-de_la_liste.add(k,"élément à la kème position");**

Exemple

```
import java.util.*;
public class maListe {

    public static void main(String args[]) {

        /*Déclaration d'une liste chaînée ( Linked List ) */
        LinkedList<String> maListe = new LinkedList<String>();

        // Ajout d'éléments à la liste
        maListe.add("Voiture");
        maListe.add("Moto");
        maListe.add("Vélo");

        /*Affichage de la liste chaînée*/
        System.out.println("Contenu de la liste avant ajout: " +maListe);

        /*Ajout d'un élément au début la liste*/
        maListe.addFirst("Avion");
        /*Ajout d'un élément à la fin de la liste*/
        maListe.addLast("miniVélo");
        /* Affichage après ajout */
        System.out.println("Contenu de la liste après ajout : " +maListe);
    }
}
```

Ce qui affiche après exécution sur Eclipse :

Contenu de la liste avant ajout: [Voiture, Moto, Vélo]

Contenu de la liste après ajout : [Avion, Voiture, Moto, Vélo, miniVélo]

3 - Récupération des élément de la liste

Pour récupérer un élément à la k ème position de la liste on utilise la syntaxe :

[nom_de_la_liste].get([numéro de position de l'élément]) ;

Voici un exemple permettant de récupérer l'élément à la 2 ème position de la liste :

Exemple

```
import java.util.LinkedList;
public class ListGet {
    public static void main(String[] args) {
        LinkedList<String> myList = new LinkedList<String>();
        myList.add("Avion");
        myList.add("voiture");
    }
}
```



```

myList.add("moto");
myList.add("Vélo");
//Accès à un élément de la liste
String s=myList.get(2);
//affichage de l'élément obtenu
System.out.println("L'élément qui se trouve à la 2ème position est : " + s);
}
}

```

Ce qui affiche après exécution sur Eclipse :

L'élément qui se trouve à la 2ème position est : moto

4 - Suppression d'un élément de la liste

4 - 1 - Suppression d'un élément par son index

Un élément quelconque de la liste peut être supprimé via son index en utilisant la syntaxe :

[nom_de_la_liste].remove(index de l'élément) ;

Voici un exemple qui permet de supprimer l'élément "moto" qui se trouve à l'index 2 de la liste

Exemple

```

import java.util.LinkedList;

public class RemoveElement {
    public static void main(String[] args) {
        LinkedList<String> myList = new LinkedList<String>();
        myList.add("Avion");
        myList.add("voiture");
        myList.add("moto");
        myList.add("Scooter");
        myList.add("Vélo");
        //affichage de la liste avant suppression
        System.out.println("Contenu de la liste avant suppression : "+myList);
        //Suppression d'un élément de la liste à une position donnée

        myList.remove(2);
        //affichage de la liste après suppression
        System.out.println("Contenu de la liste après suppression: "+myList);
    }
}

```

Ce qui affiche après exécution sur Eclipse :

Contenu de la liste avant suppression : [Avion, voiture, moto, Scooter, Vélo]
Contenu de la liste après suppression: [Avion, voiture, Scooter, Vélo]

4 - 2 - Suppression directe d'un élément par son nom

On peut aussi supprimer un élément de la liste en utilisant son nom via la syntaxe :
[nom_de_la_liste].remove([nom de de l'élément]) ;
Voici un exemple permettant de supprimer directement l'élément "Scooter" de la liste :

Exemple

```
import java.util.LinkedList;

public class Test {
    public static void main(String[] args) {
        LinkedList<String> myList = new LinkedList<String>();
        myList.add("Avion");
        myList.add("voiture");
        myList.add("moto");
        myList.add("Scooter");
        myList.add("Vélo");
        //affichage de la liste avant suppression
        System.out.println("Contenu de la liste avant suppression : "+myList);
        //Suppression directe d'un élemnt par son nom
        myList.remove("Scooter");
        //Affichage de la liste après suppression
        System.out.println("Contenu de la liste après suppression: "+myList);
    }
}
```

4 - 3 - Vider la liste

On aussi vider la liste (c.a.d supprimer tous les éléments de la liste) via la syntaxe :
[nom_de_la_liste].clear() ;

Exemple

```
import java.util.LinkedList;

public class ClearList {
    public static void main(String[] args) {
        LinkedList<String> myList = new LinkedList<String>();
        myList.add("Avion");
        myList.add("voiture");
        myList.add("moto");
    }
}
```

```

myList.add("Scooter");
myList.add("Vélo");
//Affichage de la liste avant suppression
System.out.println("Contenu de la liste avant suppression : "+myList);
// Suppression de tous les éléments de la liste
myList.clear();
// Affichage après suppression de tous les éléments
System.out.println("Contenu de la liste après clear() :");
}
}

```

Ce qui affiche après exécution sur Eclipse :

Contenu de la liste avant suppression : [Avion, voiture, moto, Scooter, Vélo]

Contenu de la liste après clear() :

5 - Parcours de la liste

5 - 1 Parcours de la liste par la méthode listIterator()

Afin de pouvoir parcourir tous les éléments de la liste via la méthode `Iterator()` , il va falloir préalablement définir la variable `iterator` qui va parcourir la liste via la syntaxe :

`ListIterator<String> it = [nom_de_la_liste].listIterator();`

On utilise ensuite la syntaxe :

```

while(it.hasNext()){
    System.out.println(it.next());
}

```

Exemple

```

import java.util.*;
public class IteratorList {
    public static void main(String[] args) {
LinkedList<String> myList = new LinkedList<String>();
myList.add("voiture");
myList.add("moto");
myList.add("Scooter");
myList.add("Vélo");
//Création de l'iterator
ListIterator<String> it = myList.listIterator();
//parcours des éléments de la liste
while(it.hasNext()){
    System.out.println(it.next());
}
}
}

```

Ce qui affiche après exécution sur Eclipse :

voiture
moto
Scooter
Vélo

5 - 2 -Parcours de la liste à l'aide de la boucle for

Nous pouvons aussi parcourir les éléments de la liste en définissant une variable qui aura pour fonction d'explorer la liste du début jusqu'à la fin en utilisant la boucle for :

```
for(String s : nom_de_la_liste){  
    System.out.println(s);  
}
```

Exemple

```
import java.util.LinkedList;  
  
public class ListLireFor {  
    public static void main(String[] args) {  
        LinkedList<String> myList = new LinkedList<String>();  
        myList.add("voiture");  
        myList.add("moto");  
        myList.add("Scooter");  
        myList.add("Vélo");  
  
        //parcourt des éléments de la liste avec la boucle for  
        for(String s:myList){  
            System.out.println(s);  
        }  
    }  
}
```

Ce qui doit afficher après exécution sur Eclipse :

voiture
moto
Scooter
Vélo

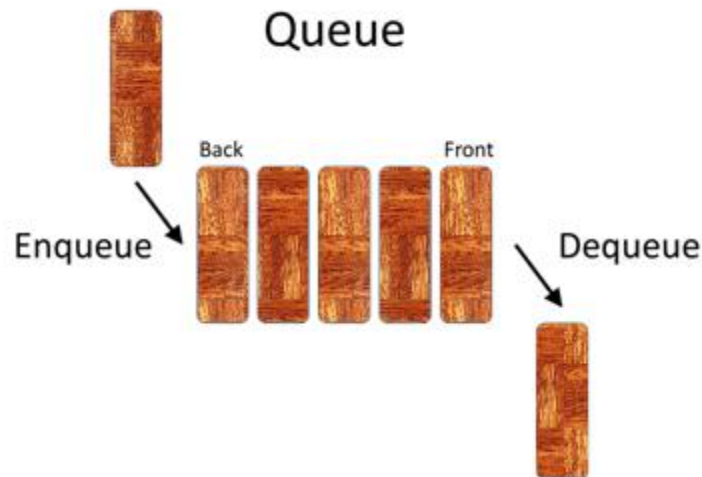
Remarque :

La même étude faite avec les variables String est valable pour les autres variables, il suffit pour cela d'indiquer au départ pendant l'instanciation le type de variable utilisé :

```
LinkedList<String> myList = new LinkedList<String>();  
LinkedList<Double> myList = new LinkedList<Double>();  
LinkedList<int> myList = new LinkedList<int>();  
// ...
```

C - Collections Java partie 3 - Queue

1 - L'interface Queue Java



Une file d'attente (Queue en anglais) est un type particulier de type de données abstraites dans lequel les entités de la collection sont conservées dans l'ordre et les opérations principales sont l'ajout d'entités à la position du terminal arrière (enqueue) et le retrait des entités de la position du terminal avant, appelée dequeue. Cela fait de la file d'attente une structure de données First-In-First-Out (FIFO). Dans une structure de données FIFO, le premier élément ajouté à la file sera le premier à être supprimé. Cela équivaut à l'exigence qu'une fois qu'un nouvel élément soit ajouté, tous les éléments qui ont été ajoutés avant doivent être supprimés avant que le nouvel élément ne puisse être supprimé.

14 - Graphique avec Java Swing

1 - Introduction au Bibliothèque Graphique Java Swing

Java Swing constitue l'une des plus récentes bibliothèque graphique et la plus utilisée pour le langage Java, Java Swing fait partie du package Java Foundation Classes (JFC) qui elle même fait partie de J2SE.

2 - Première Fenêtre avec Java Swing

Dans ce tutoriel nous allons montrer comment créer une fenêtre avec la **classe JFrame** qui se trouve dans la bibliothèque **Javax.Swing**.

Pour ce faire nous devons préalablement importer la bibliothèque **javax.swing.JFrame**; et faire ensuite l'instanciation sur la **classe JFrame** pour créer une **fenêtre**.

```
import javax.swing.JFrame;
public class maFenetre {

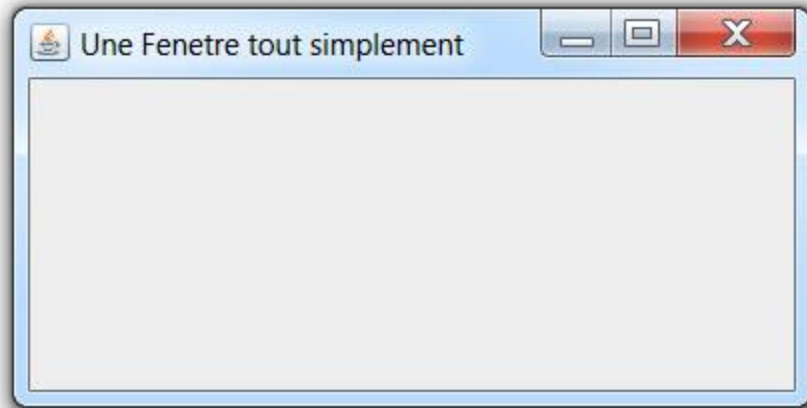
    public static void main(String[] args) {
        JFrame f=new JFrame();
    }
}
```

Ce qui n'affiche rien après exécution bien sûr, puisqu'on a pas invoquer la méthode **setVisible()** qui permet de visualiser la fenêtre. On peut donc améliorer le code en ajoutant cette dernière et celle qui définit les dimensions de la façon suivante :

```
import javax.swing.JFrame;
public class maFenetre {

    public static void main(String[] args) {
        JFrame f=new JFrame();
        f.setVisible(true);
        f.setTitle("Une Fenetre tout simplement");
        f.setSize(400,200);
    }
}
```

Ce qui affiche une fenêtre de taille 400 x 200 :



Afin de pouvoir améliorer la classe `JFrame` il vaut mieux créer une sous classe via l'instruction `extends` !

```
import javax.swing.JFrame;
public class maFenetre extends JFrame{
    //création du constructeur
    public maFenetre(){
//Titre de la fenêtre
this.setTitle("Ma propre fenêtre");
    }
}
```

Maintenant pour créer et afficher la fenêtre nous devons :

1. Créer une **méthode afficher()** permettant d'afficher la fenêtre
2. Créer une **méthode main()** pour instancier la classe **maFenetre** et afficher l'objet fenêtre

```
import javax.swing.JFrame;
public class maFenetre extends JFrame{
    //création du constructeur
    public maFenetre(){
this.setTitle("Ma propre fenêtre");
    }
    //création de la fonction d'affichage
    public void afficher(){
        this.setVisible(true);
    }
    public static void main(String[] args) {
        maFenetre f=new maFenetre();
        f.afficher();
    }
}
```

En exécutant ce code vous obtenez une petite fenêtre de dimensions nulles, par ce qu'on a pas préciser ses dimensions



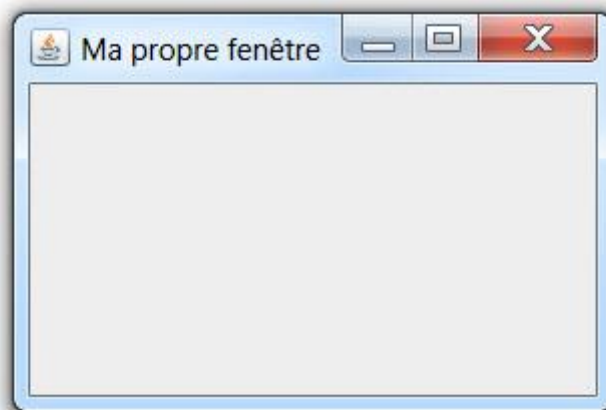
Nous pouvons maintenant améliorer le code en ajoutant au constructeur, les dimensions de la fenêtre :

```
this.setSize(largeur en pixels , hauteur en pixel);
```

Exemple

```
import javax.swing.JFrame;
public class maFenetre extends JFrame{
    //création du constructeur
    public maFenetre(){
        // Titre de la fenêtre
        this.setTitle("Ma propre fenêtre");
        // dimensions de la fenêtre :
        this.setSize(300, 200);
    }
    //création de la fonction d'affichage
    public void afficher(){
        this.setVisible(true);
    }
    public static void main(String[] args) {
        maFenetre f=new maFenetre();
        f.afficher();
    }
}
```

Ce qui affiche après exécution, une fenêtre de taille 300 x 200



3 - Les conteneurs Java Swing (Swing Containers)

Les composants Swing dans lesquels sont incorporés **les composants de la GUI** comme par exemple : **JFrame, JDialog, JPanel, JScrollPane...** sont appelé les **conteneurs Swing** (**Swing containers** en anglais)

4 - Les composants Java Swing

Les composants de la bibliothèque Swing presque identiques à ceux de Awt :

- JButton
- JLabel
- JTextArea
- JTextField
- JRadioButton
- JCheckBox
- JPasswordField
- JComboBox
- JList
- JScrollBar

5 - Ajout de composants Swing à la fenêtre

5 - 1 - Ajout d'un bouton

Avant d'ajouter un bouton à une fenêtre il faut préalablement créer un panneau (Panel) en faisant une instantiation sur la classe **JPanel** et d'associer le contenu de ce panel à la fenêtre via l'instruction :

```
fenêtre.setContentPane(panel);
```

Voici le code :

```
JFrame f=new JFrame("Ma Fenêtre");  
f.setSize(500, 400);  
JPanel pan=new JPanel();  
f.setContentPane(pan);
```

Explication : dans ce code on créer une fenêtre nommée **f** contenant un **panel** nommé **pan**

A ce moment là il est temps de créer un bouton à l'aide d'une instantiation sur la classe JButton :

```
JButton b=new JButton("Voici un Bouton");
```

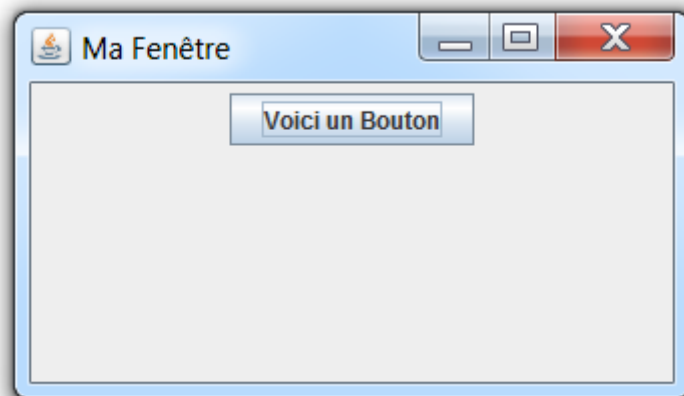
Le bouton ne sera pas visible à ce moment là ! Il va falloir l'**ajouter** à la fenêtre **f** l'aide de la méthode :

```
f.getContentPane().add(b);
```

Voici le code final :

```
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JPanel;
public class AppGaphique {
    public static void main(String[] args) {
        JFrame f=new JFrame("Ma Fenêtre");
        f.setSize(500, 400);
        JPanel pan=new JPanel();
        f.getContentPane().add(pan);
        //ajout de bouton
        JButton b=new JButton("Voici un Bouton");
        f.getContentPane().add(b);
        f.setVisible(true);
    } }
```

Ce qui affiche après exécution :



Remarque

On ajoute les autres composants **Label**, **TextField**... de la même façon en faisant une instantiation sur les classes **JLabel**, **TextField**...

6 - Les événements Java Swing

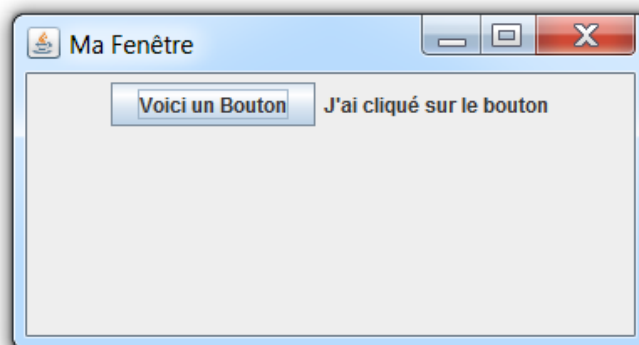
Exemple avec ActionListener appliquer à JButton

```
monBouton.addActionListener(new ActionListener(){  
    public void actionPerformed(ActionEvent e){  
        [ensemble d'actions ici ]  
    }  
});
```

Exemple

```
import java.awt.event.ActionEvent;  
import java.awt.event.ActionListener;  
import javax.swing.JButton;  
import javax.swing.JFrame;  
import javax.swing.JLabel;  
import javax.swing.JPanel;  
public class AppGaphique {  
    public static void main(String[] args) {  
        JFrame f=new JFrame("Ma Fenêtre");  
        f.setSize(500, 400);  
        JPanel pan=new JPanel();  
        f.setContentPane(pan);  
        //ajout de bouton  
        JButton b=new JButton("Voici un Bouton");  
        f.getContentPane().add(b);  
        JLabel l=new JLabel("");  
        f.getContentPane().add(l);  
        f.setVisible(true);  
        b.addActionListener(new ActionListener(){  
            public void actionPerformed(ActionEvent e){  
                l.setText("J'ai cliqué sur le bouton"); }  
        });  
    }  
}
```

Ce qui affiche après exécution et après avoir cliqué sur le bouton :



5 - Le composant JTable

Dans cette partie, nous allons voir comment créer un tableau pour afficher des données tabulaires en utilisant la classe JTable au sein de la bibliothèque Java swing. La classe **JTable** représente le **composant Swing table**. **JTable** offre une fonctionnalité riche pour gérer l'apparence et le comportement du composant table. **JTable étend directement JComponent** et **implémente** plusieurs **interfaces** d'auditeurs de modèles telles que **TableModelListener**, **TableColumnModelListener**, **ListSelectionListener** ... etc. JTable implémente également une interface défilable; par conséquent, le tableau est généralement placé dans un **JScrollPane**.

Chaque JTable comporte trois modèles :

- TableModel
- TableColumnModel
- ListSelectionModel.

TableModel : est utilisé pour spécifier comment les données de la table sont stockées et la récupération de ces données. Les données de JTable sont souvent en structure bidimensionnelle telle qu'un tableau à deux dimensions ou un vecteur de vecteurs.

TableModel est également utilisé pour spécifier comment les données peuvent être modifiées dans la table.

TableColumnModel : est utilisé pour gérer tout TableColumn en terme de sélection de colonne, d'ordre de colonne et de taille de marge.

ListSelectionModel : permet au tableau d'avoir un mode de sélection différent, tel qu'un sélecteur unique, un intervalle unique et plusieurs sélections d'intervalles.

Nous pouvons créer une table en utilisant ses constructeurs comme suit:

Constructeurs JTable	Description
JTable()	Création d'une table vide
JTable(int rows, int columns)	Création d'une table avec des lignes et des colonnes avec cellules vides.
JTable(Object[][] data, Object[] heading)	Création d'une table avec les données spécifiées du tableau bidimensionnel et l'en-tête de la colonne
JTable(TableModel dm)	Création d'une table avec le modèle TableModel
JTable(TableModel dm, TableColumnModel cm)	Création d'une table avec les modèles TableModel et TableColumnModel.
JTable(TableModel dm, TableColumnModel cm, ListSelectionModel sm)	Créer une table avec les modèles TableModel, TableColumnModel, et ListSelectionModel.
JTable(Vector data, Vector heading)	Création d'une table avec vector of Vectors <i>data</i> et column headings <i>headin</i> .

Exemple

Nous verrons ici un exemple d'utilisation de **JTable** pour afficher les achats de matériels informatiques :

- Nous commençons par spécifier l'en-tête de colonne dans le tableau des colonnes.
- Nous utilisons ensuite des données de tableau bidimensionnel pour stocker des données.
- Nous créons ensuite une instance de JTable en transmettant les données du tableau et l'en-tête de colonne au constructeur.
- Enfin, nous plaçons la table JScrollPane et l'ajoutons au cadre principal.

```
package JTableTest;

import javax.swing.*.*;
import java.awt.*.*;

public class MainClass {

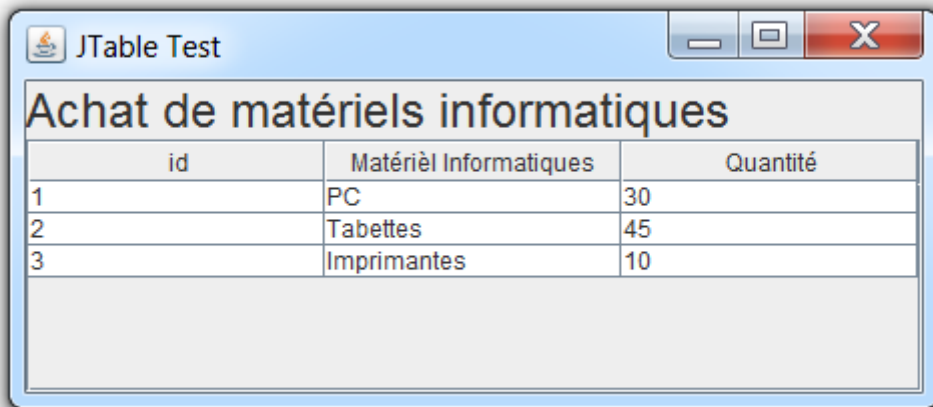
    public static void main(String[] args) {
        final JFrame frame = new JFrame("JTable Demo");

        String[] columns = {"id", "Matériel Informatiques", "Quantité"};

        Object[][] data = {
            {1,"PC ","30"},
            {2,"Tablettes","45"},
            {3, "Imprimantes", "10"}
        };

        JTable table = new JTable(data, columns);
        JScrollPane scrollPane = new JScrollPane(table);
        JLabel lblHeading = new JLabel("Achat de matériels informatiques");
        lblHeading.setFont(new Font("Arial",Font.TRUETYPE_FONT,20));
        frame.getContentPane().setLayout(new BorderLayout());
        frame.getContentPane().add(lblHeading,BorderLayout.PAGE_START);
        frame.getContentPane().add(scrollPane,BorderLayout.CENTER);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setSize(450, 250);
        frame.setVisible(true);
    }
}
```

En exécutant le code ci-dessus vous obtenez la fenêtre suivante :



A screenshot of a Java Swing window titled "JTable Test". The window has a standard Mac OS-style title bar with a red close button, a yellow maximize button, and a green minimize button. Below the title bar, the text "Achat de matériels informatiques" is displayed in a large, bold, black font. Underneath this text is a table with three columns: "id", "Matériel Informatiques", and "Quantité". The table contains three rows of data:

id	Matériel Informatiques	Quantité
1	PC	30
2	Tablettes	45
3	Imprimantes	10

Below the table, there is a large, empty rectangular area with a light gray background, likely intended for additional information or a summary.

D - Java JTextArea

La classe **JTextArea** est une classe Java permettant de créer une zone de texte multiligne. Elle permet l'édition de plusieurs lignes de texte. Il s'agit d'une classe Java qui hérite de la classe **JTextComponent**

1 - Déclaration de classe JTextArea via l'héritage de JTextComponent

```
public class JTextArea extends JTextComponent
```

2 - Constructeur de classe & description

JTextArea ()

Construit une nouvelle TextArea.

JTextArea (Document doc)

Construit un nouveau JTextArea avec le modèle de document donné, et par défaut pour tous les autres arguments (null, 0, 0).

JTextArea (Document , String texte, lignes int, colonnes int)

Construit un nouvel JTextArea avec le nombre spécifié de lignes et de colonnes, et le modèle donné.

JTextArea (int rows, int columns)

Construit une nouvelle TextArea vide avec le nombre de lignes et de colonnes spécifié.

JTextArea (String texte)

Construit une nouvelle TextArea avec le texte spécifié affiché.

JTextArea (String texte , lignes int, colonnes int)

Construit une nouvelle TextArea avec le texte et le nombre de lignes et de colonnes spécifiés.

3 - Méthodes de classe & description

void append (String str)

Ajoute le texte donné à la fin du document.

protected Document createDefaultModel ()

Crée l'implémentation par défaut du modèle à utiliser lors de la construction si aucune n'est explicitement donnée.

AccessibleContext getAccessibleContext ()

Obtient le AccessibleContext associé à cette JTextArea.

int getColumns ()

Renvoie le nombre de colonnes dans TextArea.

protected int getColumnWidth ()

Obtient la largeur de colonne.

int getLineCount ()

Détermine le nombre de lignes contenues dans la zone.

int getLineEndOffset (ligne int)

Détermine le décalage de la fin de la ligne donnée.

int getLineOfOffset (décalage int)

Traduit un décalage dans le texte des composants en un numéro de ligne.

int getLineStartOffset (ligne int)

Détermine le décalage du début de la ligne donnée.

boolean getLineWrap ()

Obtient la politique de retour à la ligne de la zone de texte.

Dimension getPreferredSize ()

Renvoie la taille préférée de la fenêtre si ce composant est incorporé dans un JScrollPane.

Dimension getPreferredSize ()

Renvoie la taille préférée de TextArea.

protected int getRowHeight ()

Définit la signification de la hauteur d'une ligne.

int getRows ()

Renvoie le nombre de lignes dans TextArea.

boolean getScrollableTracksViewportWidth()

Renvoie true si une fenêtre doit toujours forcer la largeur de ce défilement pour correspondre à la largeur de la fenêtre.

int getScrollableUnitIncrement (Rectangle visibleRect, orientation int, int direction)

Les composants qui affichent des lignes logiques ou des colonnes doivent calculer l'incrément de défilement qui va complètement exposer une nouvelle ligne ou une colonne, en fonction de la valeur de l'orientation.

int getTabSize ()

Obtient le nombre de caractères utilisés pour développer les onglets.

String getUIClassID ()

Renvoie l'ID de la classe pour l'interface utilisateur.

boolean getWrapStyleWord ()

Obtient le style de retour utilisé si la zone de texte est en train d'encapsuler des lignes.

void Insert(String str, int pos)

Insère le texte spécifié à la position spécifiée.

protected String paramString ()

Renvoie une représentation sous forme de chaîne de cette JTextArea.

void replaceRange (Chaîne str, int début, int fin)

Remplace le texte de la position de début à la fin indiquée par le nouveau texte spécifié.

void setColumns (colonnes int)

Définit le nombre de colonnes pour ce TextArea.

void setFont (Police f)

Définit la police actuelle.

void setLineWrap (enveloppe booléenne)

Définit la politique de retour à la ligne de la zone de texte.

void setRows (int rangées)

Définit le nombre de lignes pour cette TextArea.

void setTabSize (taille int)

Définit le nombre de caractères pour développer les onglets.

void setWrapStyleWord (mot booléen)

Définit le style d'habillage utilisé, si la zone de texte est en train d'encapsuler des lignes.

E - JMenu : Création des menus avec Java

1 - La barre des menus JMenuBar Java

Afin d'organiser les menus, Java est doté d'une barre susceptible d'être placée en haut de la fenêtre, permettant d'accueillir et organiser les menus Java. Pour cela le langage Java utilise une classe nommée JMenuBar est utilisée pour afficher la barre des menus dans la fenêtre JFrame.

Pour créer une barre de menu, il suffit de créer une fenêtre dotée d'un panneau (JPanel), d'instancier la classe JMenuBar, et d'ajouter ensuite la barre de menu au JPanel.

a) Création d'une fenêtre dotée d'un JPanel

```
JFrame fen=new JFrame("Exemple de fenêtre avec menu");
JPanel panel=new JPanel();
panel.setLayout(null);
fen.setSize(500, 400);
fen.setContentPane(panel);
```

b. Création de la barre des menus avec la classe JMenuBar

```
//création de la barre des menus
JMenuBar myMenuBar=new JMenuBar();
//Ajout de la barre de menus au JPanel
panel.add(myMenuBar)
```

2- La classe JMenu

La classe JMenu permet via l'instanciation, de créer un menu. Nous allons à titre d'exemple l'utiliser pour créer un menu Fichier :

```
//Création du menu Fichier
JMenu menuFichier = new JMenu("Fichier");
//Ajout du menu à la barre des menus
myMenuBar.add(menuFichier);
```

Code Complet

```
import javax.swing.JFrame;
import javax.swing.JMenu;
import javax.swing.JMenuBar;
import javax.swing.JPanel;
```

```

public class MyJMenu {

    public static void main(String[] args) {

        //création d'une fenêtre JFrame
        JFrame fen=new JFrame("Exemple de fenêtre avec menu");
        JPanel panel=new JPanel();
        //création d'un panneau (JPanel)
        panel.setLayout(null);
        fen.setSize(500, 400);
        fen.setContentPane(panel);

        //création de la barre des menus
        JMenuBar myMenuBar=new JMenuBar();

        //Ajout de la barre de menus au JPanel
        panel.add(myMenuBar);

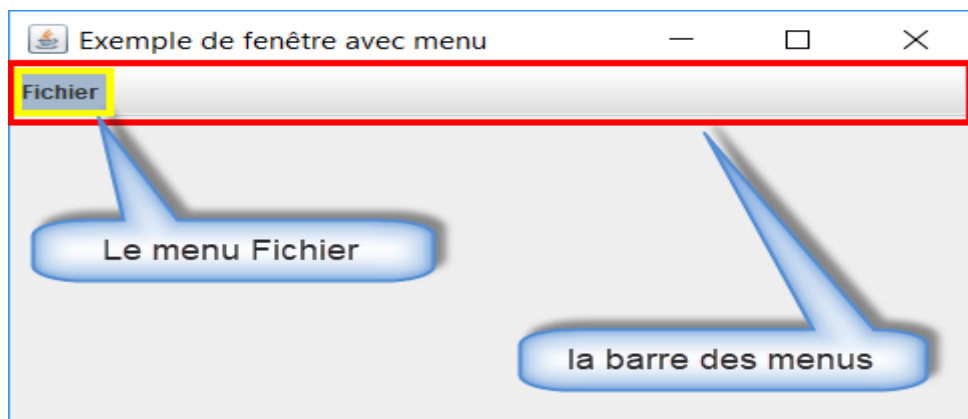
        //Positionnement de la barre des menus avec la m"thode
        myMenuBar.setBounds(0, 0, 500, 30);
        //Création du menu Fichier
        JMenu menuFichier = new JMenu("Fichier");

        //Ajout du menu à la barre des menus
        myMenuBar.add(menuFichier);
        fen.setVisible(true);

    }
}

```

Voici un aperçu après exécution du code sur Eclipse



3 – Création des sous menus avec la classe JMenuItem

Pour créer un sous menu et l'ajouter à menu principal, il suffit d'instancier la classe **JMenuItem** et appliquer la **méthode add()**. La classe JMenuItem permet de créer des sous menus, ces sous menus peuvent être ajoutés au menu principal grâce à la méthode add(). Nous allons traiter un exemple simple qui permet d'insérer le menu "**Nouveau**" au menu principal "**Fichier**" :

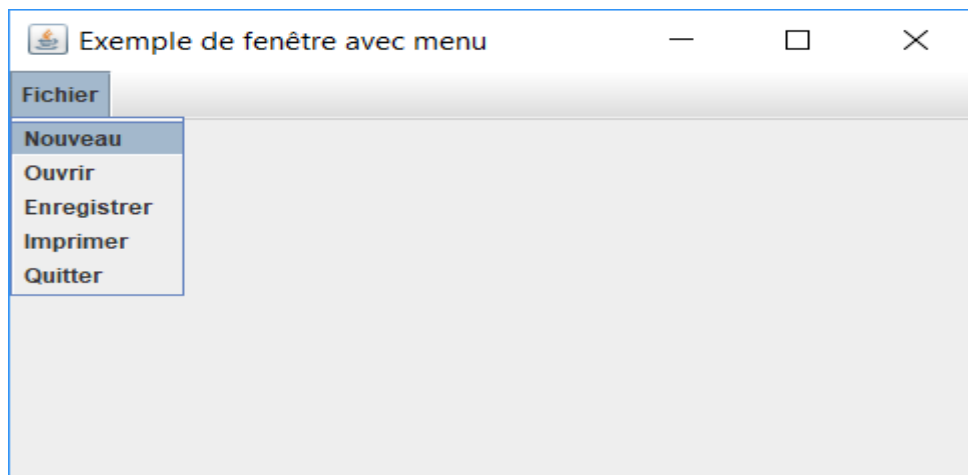
```
//Ajout d'un sous menu "Nouveau"  
JMenuItem menuNouveau = new JMenuItem("Nouveau");  
//Ajout du sous menu "Nouveau" au menu principal Fichier  
menuFichier.add(menuNouveau);
```

On peut également via la même méthode, ajouter d'autres sous menus : "**Ouvrir**" "**Enregistrer**" "**Imprimer**" "**Quitter**"

Exemple

```
//Création des sous menus  
  
JMenuItem menuNouveau = new JMenuItem("Nouveau");  
JMenuItem menuOuvrir = new JMenuItem("Ouvrir");  
JMenuItem menuEnregistrer = new JMenuItem("Enregistrer");  
JMenuItem menuImprimer = new JMenuItem("Imprimer");  
JMenuItem menuQuitter = new JMenuItem("Quitter");  
  
//Ajout des sous menus au menu principal Fichier  
menuFichier.add(menuNouveau);  
menuFichier.add(menuOuvrir);  
menuFichier.add(menuEnregistrer);  
menuFichier.add(menuImprimer);  
menuFichier.add(menuQuitter);
```

Voici un aperçu de la fenêtre obtenue après exécution du code sur Eclipse



4 - Ajouter une action à un sous menu

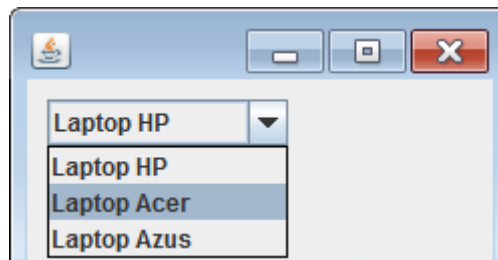
Dans le paragraphe précédent, vous avez appris à créer des menus et des sous menus, et rien d'autre ! Mais quand vous cliquez sur l'un de ces sous menus, il doit se produire une **action** ou un **événement** : exemple nous allons voir comment associer l'action qui permet de fermer la fenêtre au sous menu "Quitter".

Pour cela nous allons utiliser la même méthode **addActionListener()** que nous avons appliqué aux boutons de commandes, et au sein de cette dernière, nous devons appliquer la méthode **dispose()** pour fermer la fenêtre :

```
menuQuitter.addActionListener(new ActionListener() {  
    public void actionPerformed(ActionEvent e) {  
        fen.dispose();  
    }  
});
```

F - La liste JComboBox Java

ComboBox est un composant Swing qui affiche une liste déroulante de choix et permet à l'utilisateur de sélectionner un élément de la liste. Voici quelques captures d'écran de ce composant dans l'apparence Java par défaut sous Windows:



1 - Création d'un JComboBox Java

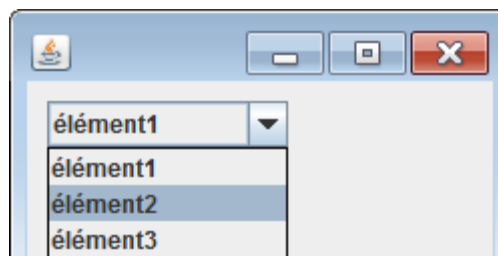
Etape 1 : Avant de créer un JComboBox, il va falloir préalablement savoir qu'il s'agit d'un composant Swing.

Pour cela, on doit impérativement créer une fenêtre dotée d'un JPanel en instanciant les classes **JFrame** et **JPanel**:

```
JFrame f=new JFrame();  
JPanel pan=new JPanel();  
f.setContentPane(pan);  
pan.setLayout(null);  
f.setSize(250,200);
```

Etape 2 : Création des éléments qui vont constituer la JComboBox. Les éléments qui vont constituer la JComboBox, peuvent être créés et organisés sur un tableau d'objets

```
Object[] elements = new Object[]{"élément1", "élément2", "élément3"};
```



Exemple :

```
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

import javax.swing.JComboBox;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JPanel;

public class ComboBox1 {

    public static void main(String[] args) {

        //- 1) --- création d'une fenêtre JFrame Java
        JFrame f=new JFrame();
        JPanel pan=new JPanel();
        f.setContentPane(pan);
        pan.setLayout(null);
        f.setSize(250,200);

        //- 2) --- définir les élément de la liste
        JComboBox<Object> elements = new JComboBox[]{"Laptop HP", "Laptop Acer", "Laptop Azus"};

        //- 3) --- Création d'une nouvelle liste
        JComboBox<Object> liste = new JComboBox(elements);
        liste.setBounds(10, 10, 120, 23);
        pan.add(liste);
        f.setVisible(true);
    }
}
```

3 - Ajout d'actions à la liste JComboBox

Après avoir créer une liste **JomboBox**, on peut lui associer une action qui se declenche au moment de la sélection à l'aide de la méthode **addActionListener()**. L'action ne peut être ajouter que si la sélection est récupérée via la méthode **getSelectedItem()**.

```
// - 4) --- Ajout d'une action à la sélection
liste.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        e.getSource();
    }
});

//- 5) --- récupération de la sélection sur une variable du type String
String s=(String) liste.getSelectedItem();
System.out.println("Vous avez sélectionné : "+s);
}
```

On peut également afficher les résultats sur un **JLabel** :

Exemple complet :

```
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

import javax.swing.JComboBox;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JPanel;

public class ComboBox2 {

    public static void main(String[] args) {

        //- 1) --- define new Window JFrame
        JFrame f=new JFrame();
        JPanel pan=new JPanel();
        f.setContentPane(pan);
        pan.setLayout(null);
        f.setSize(400,300);

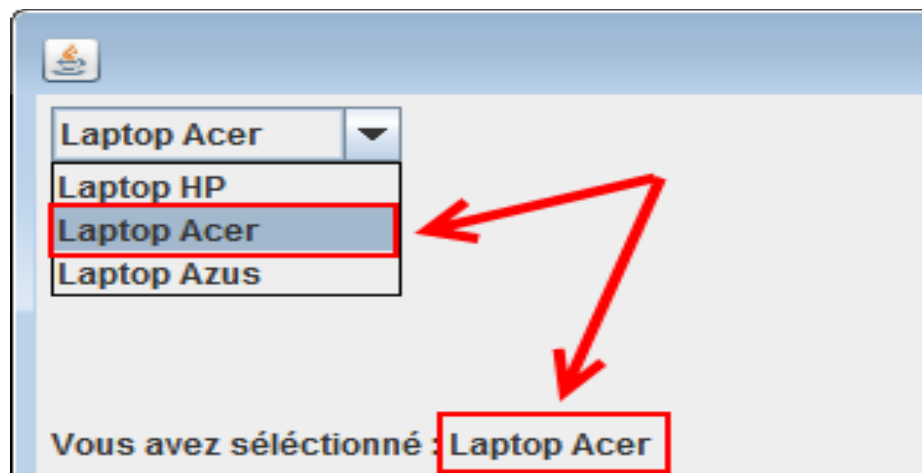
        //- 2) --- Création des éléments de la JComboBox Java
        Object[] elements = new Object[]{"Laptop HP", "Laptop Acer", "Laptop Azus"};

        //- 3) --- Création d'une nouvelle liste JComboBox
        JComboBox<String> liste = new JComboBox(elements);
        liste.setBounds(10, 10, 120, 23);
        pan.add(liste);

        //- 4) --- Création d'un JLabel pour afficher les résultats de la sélection
        JLabel label=new JLabel("View results here !");
        pan.add(label);
        label.setBounds(10,30,250,50);

        //- 5) --- appliquer une action listener à la liste JComboBox
        liste.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                e.getSource();
                String s=(String) liste.getSelectedItem();
                label.setText("Vous avez sélectionné : "+s);
            }
        });
        f.setVisible(true);
    }
}
```

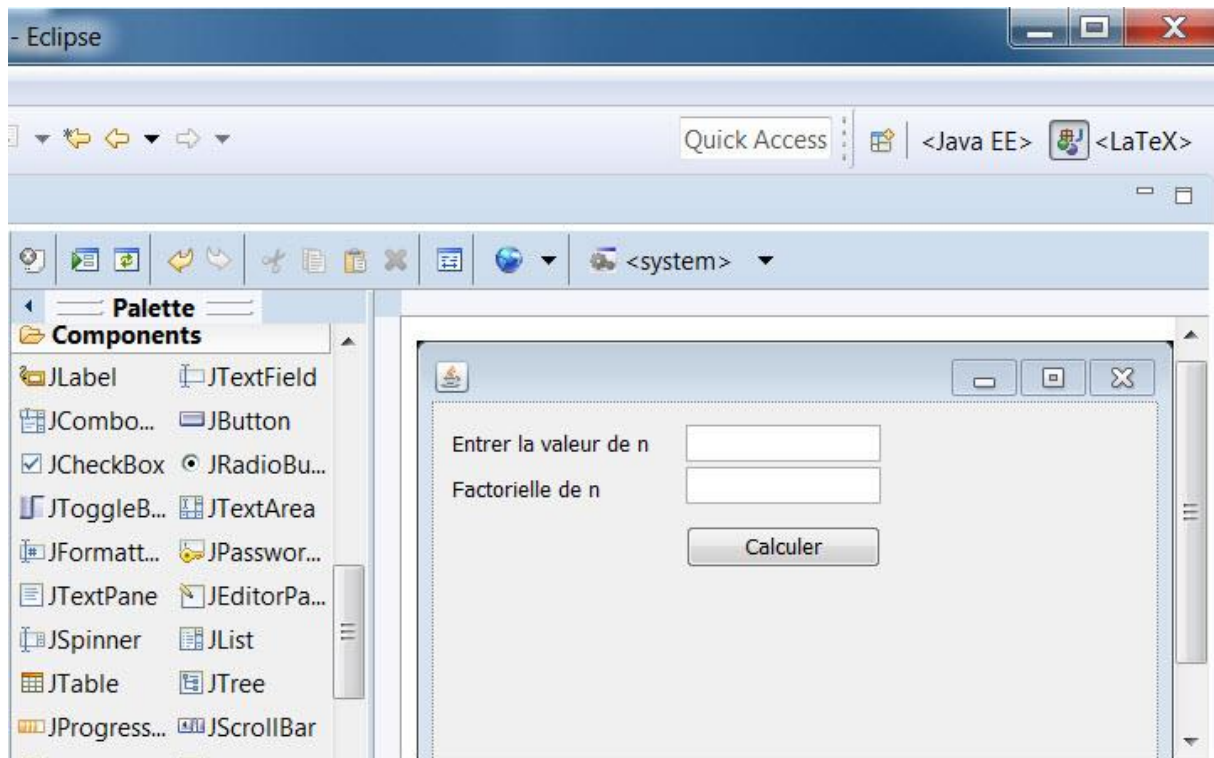
Ce qui affiche après exécution :



G - Interface graphique avec WindowBuilder

1 - Introduction

Le plugin Eclipse WindowBuilder est un concepteur Java GUI visuel, puissant et facile à utiliser permettant la création d'applications GUI Java sans vous casser la tête à écrire du code pour afficher des objets graphiques simples comme fenêtres, bouton de commandes, champs de textes... Avec WindowBuilder, vous pouvez créer des fenêtres compliquées en quelques minutes, il suffit d'utiliser le concepteur visuel et le code Java sera automatiquement généré pour vous. Vous pouvez facilement ajouter des contrôles à l'aide de glisser-déposer, gérer les événements de vos contrôles, modifier diverses propriétés des contrôles à l'aide d'un éditeur de propriétés et bien plus encore. Le code généré ne nécessite aucune bibliothèque personnalisée supplémentaire pour compiler et exécuter: l'ensemble du code généré peut être utilisé sans installer WindowBuilder.

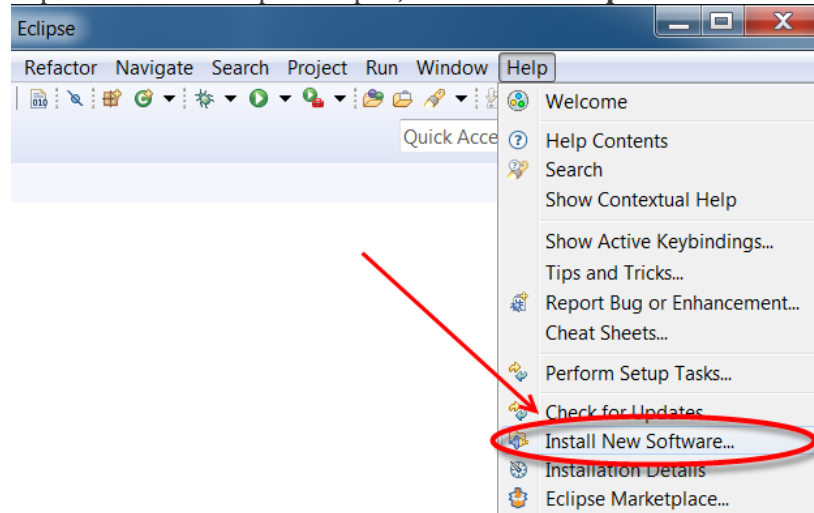


- L'éditeur est doté des principaux composants interface utilisateur (user interface) suivants:
 - Design View - la zone de présentation visuelle principale.
 - Source View- code d'écriture et analyse du code généré
 - Structure View- composée de l'arbre de composant et du volet Propriété.
 - Component Tree- montre la relation hiérarchique entre tous les composants.
 - Property Pane - affiche les propriétés et les événements des composants sélectionnés.
 - Palette - permet un accès rapide aux composants spécifiques à une trousse d'outils.

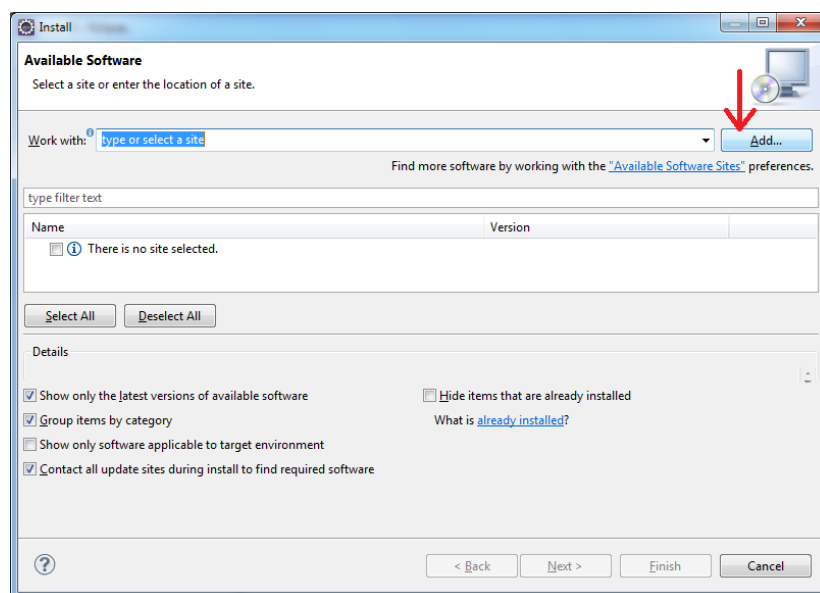
- ToolBar - permet d'accéder aux commandes couramment utilisées.
- Context Menu - permet d'accéder aux commandes couramment utilisées.

2 – Installation de WindowBuilder

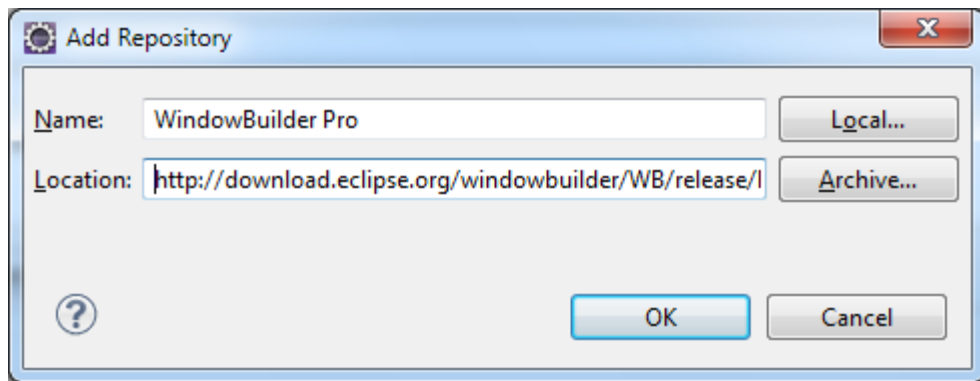
1. Copiez le lien approprié pour votre version d'Eclipse à partir de la colonne Site de mise à jour sur la page de téléchargement:
2. Depuis le menu Help d'Eclipse, choisissez **Help > Install new Software...**



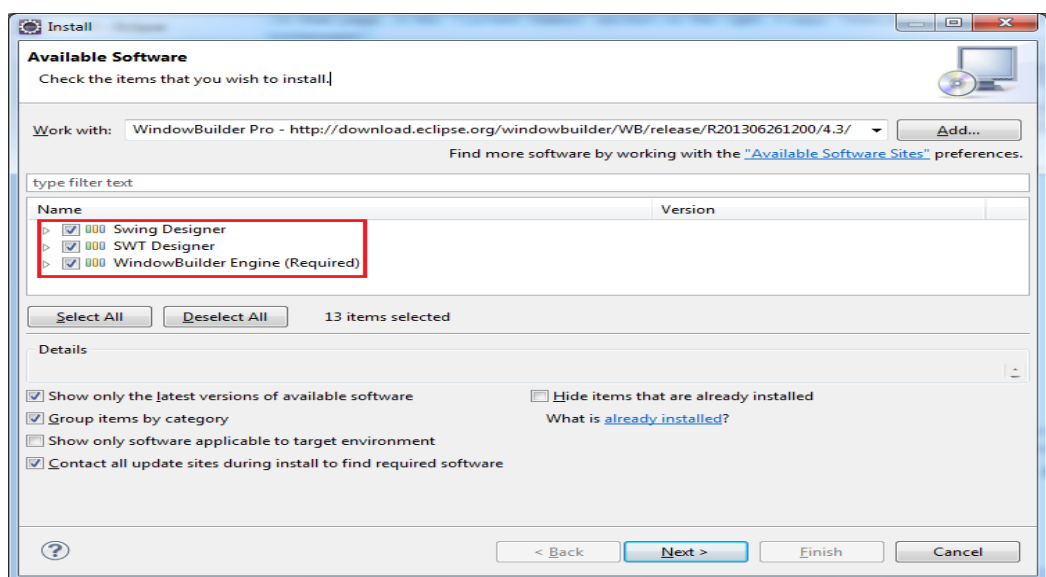
3. Dans la fenêtre qui apparaît cliquez sur **le bouton Add**



4. Dans la boîte de dialogue apparue, dans le champ Name, saisissez un nom descriptif (comme "WindowBuilder Pro") et collez le lien correct (voir étape 1) dans le champ Emplacement. Cliquez ensuite sur le bouton OK:



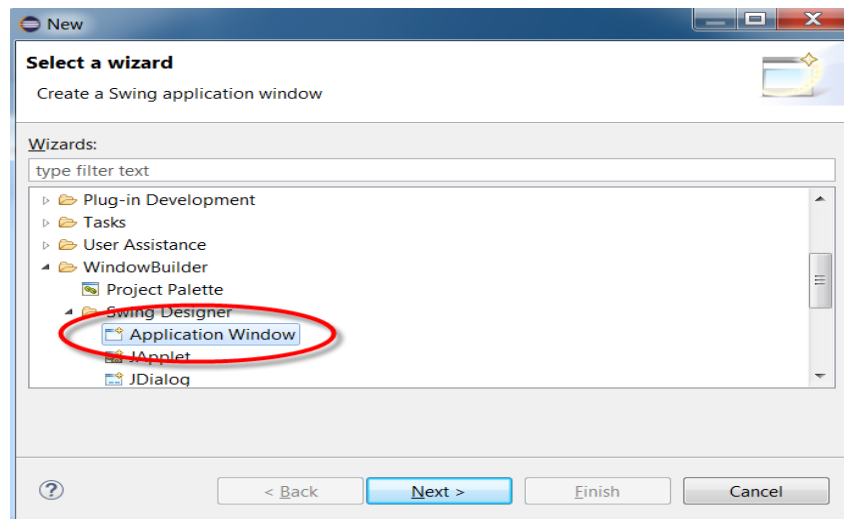
5. Sélectionnez toutes les **cases à cocher** qui vont apparaître, puis cliquez sur **Suivant** pour installer WindowBuilder :



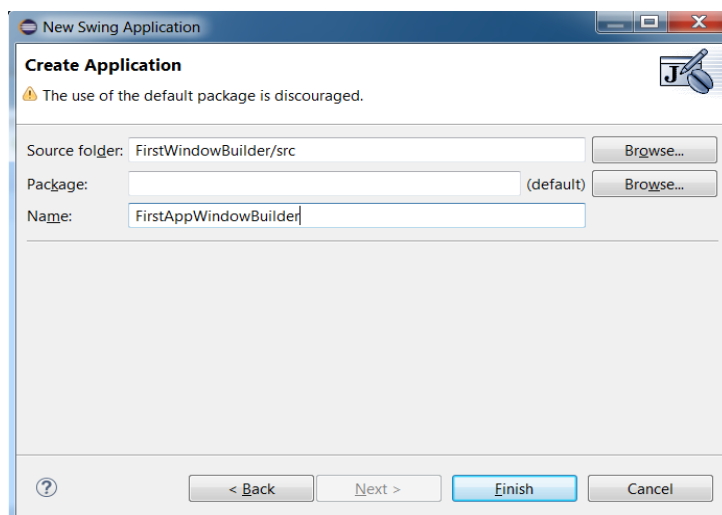
3 - Comment utiliser WindowBuilder ?

Pour utiliser WindowBuilder :

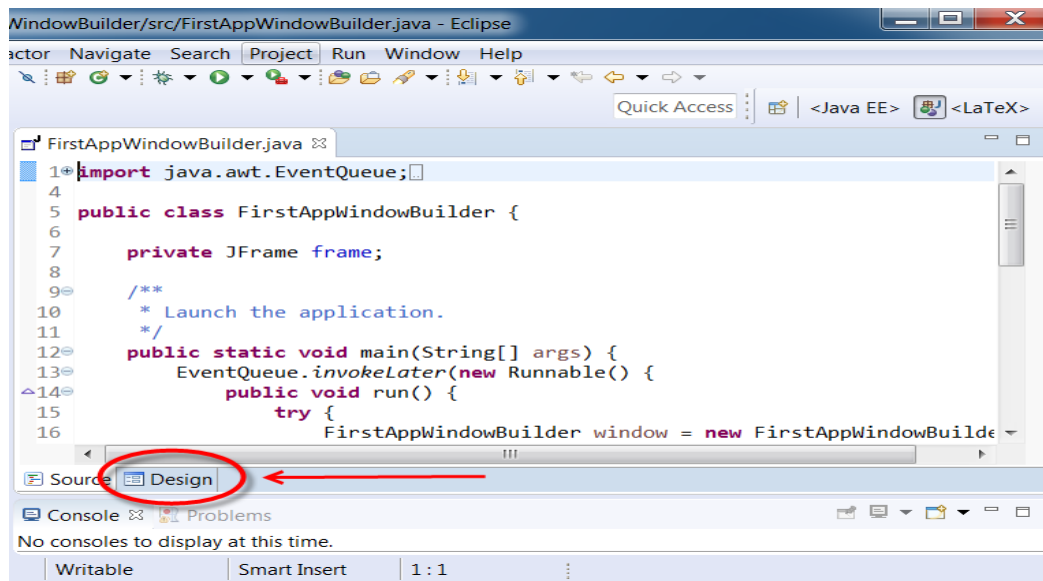
- Tapez la combinaison de touche **Ctrl + N** et vous allez voir apparaître la fenêtre suivante :



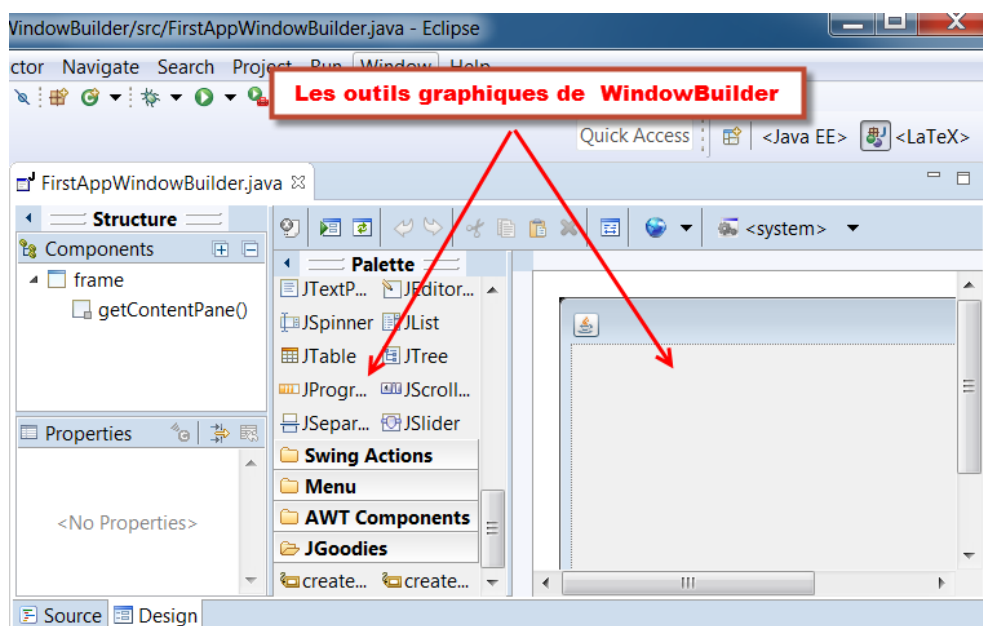
- Choisissez ensuite comme type de projet : **WindowBuilder -> Application Window**



- Après avoir cliqué sur le bouton **Finish** vous obtenez votre **premier projet WindowBuilder** :



- Cliquez maintenant sur l'onglet **Design** pour afficher la fenêtre créée automatiquement par WindowBuilder, ainsi que les autres outils graphiques WYSIWYG : Containers, Layouts, Composants Swing et Awt ...



H - Java IO- Gestion des fichiers en Java



1 - La classe File Java

La **classe File Java** permet de gérer les fichiers, les noms de chemins de fichiers et de répertoires de manière abstraite. Cette classe est utilisée pour la création de fichiers et de répertoires, la recherche de fichiers, la suppression de fichiers, etc. Nous verrons plus loin dans les prochains tutoriels que de nombreuses méthodes sont associées à cette classe. Dans ce tutoriel, nous verrons comment créer un fichier en Java à l'aide de la classe **File** et la méthode **createNewFile()**.

Cette méthode crée un fichier vide, s'il n'existe pas à l'emplacement spécifié et renvoie **true**. Si le fichier est déjà présent, cette méthode renvoie **false**.

IOException : Si une erreur d'entrée / sortie survient lors de la création du fichier.

SecurityException : Si un gestionnaire de sécurité existe et que sa méthode *SecurityManager.checkWrite (java.lang.String)* refuse l'accès en écriture au fichier.

Pour faire simple, nous allons essayer de créer un simple fichier texte sur le bureau. Pour ce faire :

1. On doit préalablement importer les bibliothèques **File** et **IOException** qui se trouvent dans le **package java.io**
2. On fait une instanciation sur la **classe File** pour créer un nouveau **objet myFile** du type **file**
3. On crée le fichier avec la méthode **createNewFile()** (**myFile.createNewFile()**)

2 - Création d'un fichier avec la classe File Java

Le code ci-dessous créerait un fichier txt nommé **newfile.txt** dans le **bureau** de l'utilisateur **acer**. Vous pouvez modifier le chemin dans le code ci-dessous afin de créer le fichier dans un répertoire ou un lecteur différent.

```

import java.io.File;
import java.io.IOException;

public class NewFile
{
    public static void main( String[] args )
    {
        try {
            File myFile = new File("C:/Users/acer/Desktop/newFile.txt");
            /* Si le fichier est créé, alors la méthode createNewFile() renverrait
            true et si le fichier est déjà présent il retournerait faux */

            boolean varF = myFile.createNewFile();
            if (varF){
                System.out.println("Fichier crée avec succès");
            }
            else{
                System.out.println("Le fichier existe déjà");
            }
        } catch (IOException e) {
            System.out.println("Exception Occurred:");
            e.printStackTrace();
        }
    }
}

```

3 - Ecrire dans un fichier existant à l'aide de la classe FileWriter

La classe FileWriter permet d'ouvrir un flux d'écriture sur un fichier via son constructeur :

FileWriter(String nom_du_fichier, boolean append)

1. Si la valeur du boolean append est mis à false, le contenu du fichier sera écrasé par le nouveau contenu
2. Si la valeur du boolean append est mis à true, le nouveau contenu sera ajouté à l'ancien

Exemple

```

try
{
    String filename= "C:/Users/acer/Desktop/newFile.txt";
    FileWriter fw = new FileWriter(filename,true);
    //la valeur true entraine l'ajout du nouveau contenu à l'ancien
    fw.write("Voici le contenu qui va être ajouté au fichier");//appends the string
to the file
    fw.close();
}
catch(IOException ioe)
{

```



```
        System.err.println("IOException: " + ioe.getMessage());
    }
}
```

4 - Supprimer un fichier existant avec la méthode delete()

Pour supprimer un fichier existant en Java, il suffit d'instancier la classe **FileWriter** et pointer vers le fichier, et appliquer ensuite la méthode **delete()** à l'objet :

```
String filename= "C:/Users/acer/Desktop/newFile.txt";
File myFile = new File(filename);
myFile.delete();
```